DCC060 - Banco de Dados

Relatório de Trabalho Prático — Entrega E1

Identificação da Equipe

Campo	Informação
Tema	Sistema de Roteamento de Entregas
SGBD utilizado	PostgreSQL
Integrante 1	João Vitor Almeida Queiroz - 202435007
Integrante 2	Arthur de Souza Marques - 202435015
Integrante 3	Gustavo Bersan Moreira Campos - 202435019
Integrante 4	Murilo Lacerda Dutra - 202435023
Integrante 5	Theo Carvalho de Faria - 202435037

1. INTRODUÇÃO

Sistema de Roteamento e Intermediação de Pedidos

Contexto do Projeto

A plataforma opera no segmento de logística de entregas e comércio eletrônico, funcionando como um marketplace que conecta lojas a consumidores através de uma rede de entregadores parceiros. O destaque do projeto é a separação entre os processos comerciais e logísticos.

Esses dois domínios são autônomos entre si. Quando um pedido é confirmado, ele gera automaticamente entregas e rotas independentes. Isso permite que cada área evolua sem afetar a outra, deixando espaço para a plataforma escalar sem quebrar o sistema.

Domínio de Aplicação

O sistema centraliza o catálogo de produtos das lojas, captura a intenção de compra, processa a divisão financeira das transações e coordena toda a distribuição através de profissionais autônomos. A plataforma também mantém controle rigoroso sobre cada mudança de estado, garantindo integridade nas transações comerciais e logísticas.

O sistema impede pedidos que misturam produtos de diferentes lojas. Todos os itens de uma compra devem pertencer ao catálogo da mesma loja, o que garante consistência e facilita o gerenciamento por parte do lojista.

A plataforma também mantém históricos de avaliação separados para lojas e entregadores, permitindo que os consumidores avaliem a qualidade e a confiabilidade de ambos.

Usuários do Sistema

O projeto foi pensado para três perfis primários de usuários:

- **Clientes**: exploram as lojas disponíveis, montam pedidos, acompanham o rastreio de suas encomendas e avaliam o serviço. As avaliações funcionam em escala de 1 a 5 estrelas, com opção de comentário textual, permitindo feedback detalhado sobre sua experiência com a loja e o entregador.
- **Lojistas**: gerenciam catálogos e preços, controlam o funcionamento de seus estabelecimentos e atualizam o status de preparação dos pedidos conforme progridem.
- **Entregadores**: visualizam a fila de entregas, assumem rotas de até duas entregas e atualizam o status logístico até a entrega final.

Problemas Resolvidos

Sistemas de delivery convencionais enfrentam três problemas principais: roteamento ineficiente, falta de transparência nos repasses financeiros e rastreabilidade deficiente.

Este projeto resolve o roteamento limitando cada rota a no máximo duas entregas simultâneas e impedindo que entregadores retenham pedidos enquanto têm entregas em atraso. Dessa forma, a plataforma garante melhor distribuição de carga e menos congestionamento na rede de entregas.

A divisão de pagamentos é automatizada assim que a transação é concluída. O sistema calcula e separa de forma transparente a fatia do lojista, a comissão do entregador e a taxa da plataforma, eliminando disputas e atrasos.

A rastreabilidade é garantida através de registros de auditoria imutáveis. Qualquer mudança de fase em um pedido ou entrega cria um histórico permanente que não pode ser alterado ou deletado, assegurando total transparência ao consumidor e segurança jurídica à plataforma.

Escopo e Delimitações

O que fica de fora do modelo, para permitir focar no que é essencial para o sistema:

- A gestão de estoque das lojas não foi incluída. A plataforma garante que produtos estejam disponíveis no catálogo no momento da venda, mas não controla inventário físico, insumos ou fichas técnicas das lojas.
- O rastreamento das entregas em tempo real (GPS) também ficou de fora. Um banco de dados relacional não é adequado para absorver coordenadas a cada segundo. O acompanhamento se limita a eventos logísticos (mudanças de status).

- A gestão de frotas e vínculos trabalhistas não foi pensada para o escopo atual. Como os entregadores são autônomos, o sistema se restringe apenas ao pagamento por corrida bem-sucedida.
- Criação de promoções como cashback e preços dinâmicos também foram deixadas de fora. O foco está no fluxo simples de compra, entrega e divisão de pagamentos.

Quanto às validações e integridades de dados, o formato dos documentos de identificação (CPF para pessoa física e CNPJ para as lojas) é garantido pelo banco de dados, que aceita apenas a quantidade correta de dígitos numéricos. O cálculo dos dígitos verificadores é feito na camada de aplicação, antes da gravação. Já os valores de preços e quantidades devem ser sempre maiores que zero, o que é garantido por restrições no banco de dados.

2. REQUISITOS DO PROJETO

Esta seção apresenta os requisitos funcionais do sistema (o que a plataforma deve oferecer aos seus usuários) e as regras de negócio (as restrições e condições que o sistema deve tratar). Para cada regra, indica-se onde ela será implementada: restrição de integridade no DDL, gatilho ou código da aplicação.

2.1 Requisitos do Sistema

ID	Requisito do Sistema
RF01	O sistema deve permitir que os Clientes gerenciem seus dados pessoais como endereço e contatos, o Lojista deve conseguir gerenciar os dados da sua loja e o entregador gerenciar seus dados e contatos.
RF02	O sistema deve permitir que o Cliente crie um Pedido, adicionando Produtos com suas quantidades específicas
RF03	O Lojista confirma a aceitação de um Pedido já pago, liberando-o para a etapa logística
RF04	O sistema deve permitir que um Entregador visualize as entregas ativas disponíveis e aceite assumir uma delas.
RF05	O sistema deve permitir a consulta da linha do tempo (rastreio passo a passo) de um Pedido e de sua Entrega.
RF06	O sistema deve permitir que o Lojista atualize o status de preparação do pedido (ex.: “Em Preparação”, “Pronto”) e que o Entregador atualize o status da logística (ex.: “A Caminho”, “Entregue”).

ID	Requisito do Sistema (cont.)
RF07	O sistema deve permitir que o Lojista altere o status de funcionamento da sua loja (ex.: “Aberta”, “Fechada”) e pause a venda de produtos específicos do catálogo sem precisar excluí-los do banco.
RF08	O sistema deve permitir que o Cliente envie uma Avaliação (nota e comentário) direcionada à Loja e ao Entregador.
RF09	O sistema deve calcular e registrar a divisão financeira do valor pago, discriminando a fatia devida à Loja, a comissão do Entregador e a taxa retida pela plataforma.
RF10	O sistema deve fornecer consultas parametrizadas para as Lojas (ex.: filtrar por datas e status) para listar o volume de pedidos concluídos e o faturamento.

2.2 Regras de Negócio

ID	Regra de Negócio	Implementação
RN01	Um pedido só pode ser feito se a loja estiver aberta.	Aplicação
RN02	Uma rota deve ser composta por no máximo duas entregas.	Gatilho
RN03	Caso exista alguma entrega atrasada, o entregador fica impedido de assumir uma segunda entrega.	Aplicação
RN04	Qualquer mudança nos status do pedido ou da entrega deve gerar uma linha na respectiva tabela de histórico.	Gatilho
RN05	A soma da parcela da loja, da comissão do entregador e da taxa da plataforma deve ser igual ao valor total pago pelo pedido.	Restrição (DDL)
RN06	A nota dada na Avaliação (para a Loja ou para o Entregador) deve ser obrigatoriamente um número inteiro de 1 a 5.	Restrição (DDL)
RN07	Um pedido não pode misturar Produtos de lojas diferentes. Todos os itens do pedido devem pertencer ao catálogo da mesma Loja.	Restrição (DDL)

ID	Regra de Negócio (cont.)	Implementação
RN08	Os registros gravados nas tabelas de Histórico de Status são imutáveis. O sistema deve bloquear qualquer tentativa de UPDATE ou DELETE nestas tabelas.	Gatilho
RN09	O valor unitário de um Produto no catálogo e a quantidade comprada no Item do Pedido devem ser estritamente maiores que zero.	Restrição (DDL)
RN10	O CPF (11 dígitos) e o CNPJ (14 dígitos) devem conter apenas números e ter a quantidade correta de dígitos.	Restrição (DDL)
RN11	O CPF e o CNPJ devem ter dígitos verificadores válidos (cálculo estrutural) antes de serem gravados.	Aplicação
RN12	Uma Entrega só pode ser criada para um Pedido já pago pelo Cliente e confirmado pela Loja e Pronto	Gatilho
RN13	Uma Avaliação só pode ser registrada para um Pedido cuja Entrega esteja finalizada.	Gatilho

2.2.1 Esquema relacional

PESSOA(**id_pessoa**, nome, cpf, email)

CLIENTE(**id_pessoa**, id_endereco_atual)
id_pessoa *References* PESSOA.id_pessoa
(id_pessoa, id_endereco_atual) *References* PESSOA_ENDERECO(id_pessoa, id_endereco)

LOJISTA(**id_pessoa**)
id_pessoa *References* PESSOA.id_pessoa

ENTREGADOR(**id_pessoa**, tipo_veiculo)
id_pessoa *References* PESSOA.id_pessoa

TELEFONE(**id_pessoa**, **numero**, tipo)
id_pessoa *References* PESSOA.id_pessoa

ENDERECO(**id_endereco**, logradouro, numero, bairro, cep, cidade, estado)

PESSOA_ENDERECO(**id_pessoa**, **id_endereco**, apelido, complemento, referencia)
id_pessoa *References* PESSOA.id_pessoa
id_endereco *References* ENDERECO.id_endereco

LOJA(**id_loja**, id_lojista, id_endereco, complemento_endereco, cnpj, nome, telefone, status_funcionamento, tempo_entrega_estimado, taxa_entrega)
id_lojista *References* LOJISTA.id_pessoa
id_endereco *References* ENDERECO.id_endereco

PRODUTO(**id_produto**, id_loja, nome, descricao, preco, ativo)
id_loja *References* LOJA.id_loja

CATEGORIA(**id_categoria**, id_loja, nome)
id_loja *References* LOJA.id_loja

PRODUTO_CATEGORIA(**id_produto**, **id_categoria**, id_loja)
(id_loja, id_produto) *References* PRODUTO(id_loja, id_produto)
(id_loja, id_categoria) *References* CATEGORIA(id_loja, id_categoria)

PEDIDO(**id_pedido**, id_loja, id_cliente, id_endereco, complemento_entrega, referencia_entrega, observacao_entrega, status, data_pedido, forma_pagamento, subtotal, taxa_entrega, taxa_servico, valor_total)
id_loja *References* LOJA.id_loja
id_cliente *References* CLIENTE.id_pessoa
id_endereco *References* ENDERECO.id_endereco

ITEM_PEDIDO(**id_pedido**, **id_produto**, id_loja, quantidade, valor_unitario)
(id_loja, id_pedido) *References* PEDIDO(id_loja, id_pedido)
(id_loja, id_produto) *References* PRODUTO(id_loja, id_produto)

ROTA(**id_rota**, id_entregador, criada_em, encerrada_em)
id_entregador *References* ENTREGADOR.id_pessoa

ENTREGA(**id_entrega**, id_pedido, id_rota, status, previsao_entrega)
id_pedido *References* PEDIDO.id_pedido
id_rota *References* ROTA.id_rota

AVALIACAO_LOJA(**id_pedido**, nota, comentario, data_avaliacao)
id_pedido *References* PEDIDO.id_pedido

AVALIACAO_ENTREGADOR(**id_pedido**, nota, comentario, data_avaliacao)
id_pedido *References* PEDIDO.id_pedido

DIVISAO_PAGAMENTO(**id_pedido**, valor_total, parcela_loja, comissao_entregador, taxa_plataforma, calculada_em)
id_pedido *References* PEDIDO.id_pedido

HISTORICO_PEDIDO(**id_pedido**, **status**, data_hora
id_pedido *References* PEDIDO.id_pedido

HISTORICO_ENTREGA(**id_entrega**, **status**, data_hora)
id_entrega *References* ENTREGA.id_entrega

2.2.2 Decisões de mapeamento não óbvias

Estratégia de mapeamento da generalização: A generalização de PESSOA em CLIENTE, LOJISTA e ENTREGADOR é total e compartilhada. Adotamos o mapeamento de **uma relação por entidade**: PESSOA guarda os atributos comuns e cada papel vira uma relação cuja chave primária é também chave estrangeira para PESSOA.

Os motivos para essa escolha são três. Primeiro, no nosso sistema um usu'ario pode assumir v'arios pap'eis simultaneamente, mas seus dados cadastrais base (como nome e CPF) s ao registrados uma unica vez na relaç ao PESSOA evitando a duplicaç ao de dados. Segundo, cada papel é referenciável por chave estrangeira: PEDIDO referencia CLIENTE, LOJA referencia LOJISTA e ROTA referencia ENTREGADOR, de modo que o banco impede que uma pessoa sem o papel correspondente apareça nessas relações. Terceiro, os atributos próprios de cada papel ficam isolados, sem valores nulos nas demais tabelas.

Além da generalização, as seguintes decisões de mapeamento também justificam destaque:

- **Garantia de que os itens são da mesma loja (RN07).** Para impedir itens de lojas diferentes no mesmo pedido, usamos chaves estrangeiras compostas. Em ITEM_PEDIDO, a mesma coluna `id_loja` é obrigada a satisfazer a referência de PEDIDO e de PRODUTO simultaneamente. Se as lojas divergirem, o banco rejeita o relacionamento nativamente.
  A Mesma ideia é aplicada em PRODUTO_CATEGORIA, para que produto e categoria pertençam à mesma loja.

- **Endereço.** Os campos de ENDERECO são imutáveis e o mesmo endereço é uma única linha, compartilhada por quem mora junto. O complemento e a referência são de cada pessoa e ficam em PESSOA_ENDERECO, de modo que a edição de um morador não se propaga aos outros. O pedido grava cópia do complemento e da referência no momento da compra, para preservar o histórico.

- **Endereço atual do cliente.** A chave estrangeira composta para PESSOA_ENDERECO garante que o endereço atual seja um dos cadastrados pela própria pessoa. O atributo pode ser nulo para quem ainda não cadastrou endereço.

- **Dois ciclos de status.** O pedido e a entrega têm status e históricos próprios, preservando a autonomia entre o domínio comercial e o logístico. A entrega é criada quando o pedido fica pronto, e `id_rota` pode ser nulo até um entregador assumir a entrega.

- **Avaliações e divisão.** Usam `id_pedido` como chave primária, o que impede mais de uma de cada tipo por pedido. A loja, e o entregador avaliados são obtidos pelo pedido.

- **Divisão do pagamento.** Quando o pedido é entregue, a aplicação calcula as três parcelas. A **parcela da loja** é o subtotal menos o valor retido pela plataforma sobre

ele; a **comissão do entregador** é a taxa de entrega; e a **taxa da plataforma** é a taxa de serviço cobrada do cliente mais o valor retido sobre o subtotal. O percentual retido é um parâmetro da plataforma, igual para todas as lojas, e não um atributo do banco.

2.2.3 Restrições que não cabem no DER nem no modelo lógico

Abaixo estão as restrições e regras de negócio que não podem ser aplicadas pelo domínio estrutural do modelo relacional e DER (chaves primárias, estrangeiras e cardinalidades). As restrições estão divididas entre regras que precisam de implementação via gatilhos ou na aplicação e restrições mais simples (resolvidas por exemplo via `CHECK`).

ID	Restrição	Mecanismo
Cardinalidade e Limites Máximas		
RR01	Um LOJISTA deve possuir ao menos uma LOJA. Não é permitido excluir ou transferir sua última loja.	Gatilho Adiado
RR02	Uma ROTA deve ser composta por, no máximo, duas entregas simultâneas.	Gatilho (BEFORE INSERT)
Validações Multi-Tabelas e Pertencimento		
RR03	O `id_endereco` de um PEDIDO deve, obrigatoriamente, pertencer ao conjunto de endereços cadastrados por aquele cliente em PESSOA_ENDERECO.	Gatilho (BEFORE INSERT)
RR04	Um PEDIDO só pode ser criado se o status de funcionamento da LOJA for aberto e se os produtos estiverem ativos no catálogo.	Aplicação
RR05	Se um ENTREGADOR possuir uma entrega com status atrasado, o sistema deve bloqueá-lo de assumir novas rotas.	Aplicação
Mudança de Estados/Fases		
RR06	O status de um PEDIDO e de uma ENTREGA devem obedecer rigorosamente ao fluxo de transição predefinido (ex: *solicitado* → *aceito*), proibindo saltos indevidos.	Gatilho (BEFORE UPDATE)
RR07	A ENTREGA só pode ser criada se o PEDIDO correspondente estiver pago e com o status de preparo finalizado (*pronto*).	Gatilho (BEFORE INSERT)
RR08	AVALIACAO de (Loja/Entregador) e DIVISAO_PAGAMENTO só podem ser registradas caso a ENTREGA do pedido esteja concluída (*entregue*).	Gatilho (BEFORE INSERT)
"Prints"e Cálculos		

ID	Restrição	Mecanismo
RR09	O `valor_unitario` do item e a `taxa_entrega` no PEDIDO devem ser copiados dos valores atuais da loja e do produto no momento da compra, não podendo ser alterados via chave estrangeira.	Gatilho / Aplicação
RR10	Os valores totais armazenados em PEDIDO e DIVISAO_PAGAMENTO devem corresponder exatamente à soma dos valores unitários dos itens mais as taxas aplicáveis.	Gatilho Adiado
Imutabilidade e Historico		
RR11	Os registros nas tabelas HISTORICO_PEDIDO e HISTORICO_ENTREGA são imutáveis.	Gatilho
RR12	Dados de localização (`logradouro`, `numero`, `cep`) na tabela ENDERECO não podem sofrer alterações após sua inserção.	Gatilho (Bloqueio de UPDATE)
Restrições Simples de Domínio (CHECK)		
RR13	Valores financeiros e de quantidade (preços, taxas, quantidades de itens e parcelas) devem ser estritamente maiores ou maiores/iguais a zero, conforme seu contexto.	CHECK
RR14	Documentos de identificação (CPF, CNPJ) e códigos postais (CEP) devem possuir tamanho fixo e numérico válido (11, 14 e 8 dígitos, respectivamente). A sigla de estado (`UF`) deve ter 2 caracteres.	CHECK / Domínio
RR15	Notas de avaliação registradas para LOJA ou ENTREGADOR devem ser estritamente números inteiros no intervalo fechado de 1 a 5.	CHECK

2.3 Normalização

A análise das dependências funcionais (DFs) considerou a semântica do domínio de negócio. Destacam-se duas premissas importantes para a classificação das formas normais: primeiro, o atributo `tipo` de TELEFONE é uma classificação pessoal dada pelo usuário (celular, fixo, etc.), não sendo determinado pelo número em si; segundo, um `cep` determina de forma única uma `cidade` e um `estado` (`cep` $\rightarrow$ `cidade, estado`), gerando uma dependência funcional que impacta a estrutura de endereços.

2.3.1 Forma normal de cada relação

Relação	**Forma**	**Justificativa**
Pessoa	FNBC	Chaves candidatas `id_pessoa`, `cpf` e `email`; todo determinante é chave.
Cliente	FNBC	`id_pessoa` → `id_endereco_atual`; o determinante é a chave.
Lojista	FNBC	Possui apenas a chave primária; não há DF não trivial.
Entregador	FNBC	`id_pessoa` → `tipo_veiculo`.
Telefone	FNBC	(`id_pessoa`, `numero`) → `tipo`; sem outras DFs ocultas.
Endereco	2FN	A chave primária é simples (`id_endereco`), logo não há dependências parciais. Entretanto, `cep` → `cidade, estado`, sendo `cep` não superchave e os atributos determinados não primos, o que viola a 3FN.
Pessoa_Endereco	FNBC	(`id_pessoa`, `id_endereco`) → `apelido, complemento, referencia`.
Loja	FNBC	Chaves candidatas `id_loja` e `cnpj`; todos os atributos dependem delas.
Produto	FNBC	`id_produto` → demais atributos.
Categoria	FNBC	`id_categoria` → `id_loja, nome`.
Produto_Categoria	1FN	`id_produto` → `id_loja` e `id_categoria` → `id_loja`. Como `id_loja` depende de atributos que são partes individuais da chave composta (`id_produto`, `id_categoria`), há violação da 2FN. A redundância é mantida para garantir a integridade referencial entre produtos e categorias da mesma loja.
Pedido	FNBC	A chave primária é simples (`id_pedido`). A relação não apresenta dependências parciais, e a igualdade aritmética entre `subtotal`, `taxa_entrega`, `taxa_servico` e `valor_total` constitui uma regra de cálculo, não uma dependência funcional clássica. A manutenção de `valor_total` preserva o valor financeiro consolidado do pedido.
Item_Pedido	1FN	`id_pedido` → `id_loja` e `id_produto` → `id_loja`. Como `id_loja` depende de atributos que são partes individuais da chave composta (`id_pedido`, `id_produto`), há violação da 2FN. A redundância é mantida para garantir a integridade referencial e assegurar que todos os produtos do pedido pertençam à mesma loja (RN07).

Relação	Forma	Justificativa
Rota	FNBC	`id_rota` → `id_entregador, criada_em, encerrada_em`.
Entrega	FNBC	Chaves candidatas `id_entrega` e `id_pedido`.
Avaliacao_Loja	FNBC	`id_pedido` → `nota, comentario, data_avaliacao`.
Avaliacao_Entregador	FNBC	`id_pedido` → `nota, comentario, data_avaliacao`.
Divisao_Pagamento	FNBC	A chave primária é simples (`id_pedido`). A soma de `parcela_loja`, `comissao_entregador` e `taxa_plataforma` corresponde a uma regra de integridade aritmética, não a uma dependência funcional clássica. A manutenção de `valor_total` permite validar a divisão financeira por meio de restrições de integridade.
Historico_Pedido	FNBC	(`id_pedido`, `status`) → `data_hora`. Como um mesmo pedido não registra o mesmo status mais de uma vez, o determinante corresponde à chave primária.
Historico_Entrega	FNBC	(`id_entrega`, `status`) → `data_hora`. Como uma mesma entrega não registra o mesmo status mais de uma vez, o determinante corresponde à chave primária.

2.3.2 Caminho escolhido

Como cinco relações não estão na 3°FN, iremos seguir com o **primeiro caminho**. A relação Endereco será decomposta, porque é o único caso em que a decomposição não traz conflitos com nenhuma regra de negócio. Para as outras quatro, vamos justificar por que optamos por desnormalizar de propósito.

2.3.3 Decomposição de Endereco

Esquema original: Endereco(**id_endereco**, logradouro, numero, bairro, cep, cidade, estado).
Dependências funcionais:

- `id_endereco` → `logradouro, numero, bairro, cep, cidade, estado`
- `cep` → `cidade, estado`

Analisando a relação Endereco, como a chave primária é simples, não há dependência parcial, e a relação está em 2FN. Porém, a dependência funcional `cep` → `cidade, estado` gera uma dependência transitiva (`id_endereco` → `cep` → `cidade, estado`), pois `cep` não é chave e `cidade` e `estado` não pertencem a nenhuma chave candidata. Isso viola a 3FN e, consequentemente, a FNBC.

Dados de exemplo:

id_endereco	logradouro	numero	bairro	cep	cidade	estado
1	Rua Halfeld	150	Centro	36010-000	Juiz de Fora	MG
2	Rua Halfeld	200	Centro	36010-000	Juiz de Fora	MG
3	Av. Afonso Pena	1050	Funcionários	30130-002	Belo Horizonte	MG

Anomalias:

- *Redundância e alteração:* a cidade e o estado do CEP 36010-000 estão repetidos nas linhas 1 e 2. Corrigir o nome do município só na linha 1 deixaria o mesmo CEP em duas cidades diferentes. A imutabilidade do endereço reduz esse risco, mas não elimina a repetição física no armazenamento.
- *Inserção:* não é possível registrar que o CEP 36016-000 pertence a um município sem cadastrar um endereço completo com logradouro e número, pois essa informação só existe dentro de uma linha de ENDERECO, identificada por `id_endereco`.
- *Exclusão:* excluir a linha 3, única com o CEP 30130-002, apaga também a informação de que esse CEP específico pertence a Belo Horizonte, MG.

Decomposição:
CEP(**cep**, cidade, estado)
ENDERECO(**id_endereco**, logradouro, numero, bairro, cep)
cep *References* CEP.cep
Junção sem perdas. O atributo em comum que liga as duas tabelas é o `cep`. Como o `cep` virou a chave primária da nova relação CEP, nós garantimos que, ao fazer um JOIN para juntar os dados de volta, o banco não vai gerar linhas falsas e a tabela original será remontada perfeitamente.
Preservação de dependências. `cep` → `cidade, estado` fica em CEP,
e `id_endereco` → `logradouro, numero, bairro, cep` fica em ENDERECO.
A DF original `id_endereco` → `cidade, estado` é recuperada por transitividade.
Nenhuma DF se perde.
Forma normal resultante. Em CEP a única DF tem a chave `cep` como determinante; em ENDERECO o único determinante é a chave `id_endereco`, logo ambas estão na FNBC.

2.3.4 Desnormalizações propositais

Nas quatro relações abaixo, a decomposição é possível, preservaria as dependências e não iria apresentar perdas de junção. No entanto, aplicar a decomposição iria enfraquecer ou impedir a validação de regras de negócio por meio de restrições do SGBD.

- **Item_Pedido** (`id_pedido` → `id_loja`): A decomposição removeria o `id_loja`, atributo que também está presente em Pedido. Sua manutenção permite definir chaves estrangeiras compostas que garantem que o pedido e o produto referenciado pertencem à mesma loja. Dessa forma, a regra de que os itens de um pedido não podem pertencer a lojas diferentes (RN07) é validada por meio de DDL. A redundância não introduz inconsistência, pois as chaves estrangeiras obrigam o `id_loja` do item a corresponder simultaneamente ao do pedido e ao do produto.

- **Produto_Categoria** (`id_produto` → `id_loja`): O raciocínio é o mesmo do item anterior. A manutenção do `id_loja` permite definir chaves estrangeiras compostas que garantem que o produto e a categoria associados pertencem à mesma loja. Sem esse atributo, a validação dessa regra dependeria de um mecanismo adicional, como um gatilho.

- **Pedido** ({`subtotal, taxa_entrega, taxa_servico`} → `valor_total`): Embora o `valor_total` seja funcionalmente determinado pelos demais atributos, guardar esse atributo permite preservar o valor financeiro consolidado do pedido, evitando a necessidade de se fazer calculos a cada consulta. Esse valor será a referência utilizada no processamento da divisão de pagamentos. Sua consistência é garantida por restrições de domínio (`CHECK`) e pelo gatilho de consolidação financeira (RR10).

- **Divisao_Pagamento** ({`parcela_loja, comissao_entregador, taxa_plataforma`} → `valor_total`): A manutenção do `valor_total` na própria relação permite validar, por meio de uma restrição `CHECK`, que a soma das parcelas corresponde ao total da divisão de pagamento (RN05). Sem essa coluna/campo, a validação dependeria de um mecanismo adicional. O gatilho da RR10 garante que o total registrado nessa relação corresponda ao valor consolidado em Pedido.

Existem outras redundâncias entre relações que não configuram violações de normalização, pois representam informações de histórico preservadas de forma intencional . É o caso do `valor_unitario` copiado para Item_Pedido, que preserva o preço exato praticado no momento da compra (RR09), e dos atributos `complemento_entrega` e `referencia_entrega` em Pedido, que preservam as instruções originais de entrega, independentemente de alterações posteriores no cadastro de endereços do cliente.

2.3.5 Esquema após a normalização

O esquema da Seção de modelagem lógica permanece o mesmo, exceto pela substituição de Endereco pelo par Cep e Endereco mostrado acima.

A. Declaração de Uso de Ferramentas de IA

Seção do trabalho	Ferramenta	Para quê e o que foi revisado pela equipe
1. Introdução	Claude (Anthropic)	Revisão do texto que a equipe já havia escrito, com sugestões de correção, corrigir pontuação, remover uma frase de enchimento e alinhar paragrafos. A equipe leu cada sugestão e aplicou as que concordou.
2.1 Requisitos do sistema	Claude (Anthropic)	Apontou requisitos com regras embutidas (RF03, RF05 e RF08). A equipe decidiu manter nos requisitos só a funcionalidade e mover as condições para regras de negócio.
2.2 Regras de negócio	Claude (Anthropic)	Análise da coerência da coluna "onde será implementada" e sugestão das novas regras RN12 e RN13, além da separação de RN10 e RN11. A equipe discutiu cada ponto e tomou as decisões finais: RN02 como restrição no DDL usando uma coluna de posição, RN07 como restrição no DDL com chaves estrangeiras compostas, RN05 reformulada como restrição de soma e RN10/RN11 separadas.
3.1 DER: levantamento de entidades e relacionamentos	Gemini (Google) e Claude (Anthropic)	O Gemini nos apoiou no levantamento inicial de entidades e cardinalidades. O Claude pesquisou na web campos baseados no iFood para serem usados como referência. Para cada ponto apresentado pelas LLM's respondemos cada questão com base no funcionamento que esperamos para o sistema e decidimos todos os pontos.
3.2 Esquema relacional	Claude (Anthropic)	Proposta de atributos de todas as entidades, com pesquisa de campos de endereço, pedido e divisão do pagamento inspirados no iFood, e rascunho do esquema em LaTeX.

Continua na próxima página

Seção do trabalho	**Ferramenta**	**Para quê e o que foi revisado pela equipe**
3.2 Restrições fora do DER e do modelo lógico	Claude (Anthropic)	Nós deu uma primeira lista com 36 restrições, classificadas por mecanismo (CHECK, gatilho, gatilho adiado e aplicação). Reduzimos e reestruturamos a lista para 15 itens, agrupados por mecanismo de implementação.
3.3 Normalização	Claude (Anthropic)	Levantamento das dependências funcionais, identificação das relações fora da 3FN, e rascunho do texto. Após esse levantamento revisamos e ajustamos a justificativa da forma normal de Endereco e as desnormalizações propositais.
Formatação do relatório	Claude (Anthropic)	Geração do código LaTeX.